

Laboratorio #2 CNN

Investigación: capas de PyTorch para la CNN

- nn.Conv2d: Aplica convolución bidimensional en una señal de entrada formada por varios canales. Se usa para el procesamiento de imágenes o mapas de características en CNNs.
- nn.MaxPool2d: Capa de agrupación que reduce el tamaño de imágenes o mapas de características por medio de una ventana para guardar el valor más alto de cada zona, así bajando el uso de memoria y acelera a la red.
- nn.AvgPool2d: Capa de agrupación que reduce el tamaño espacial de una imagen (dimensiones) de una imagen o mapa de características por medio de cálculos de promedios de los valores dentro de una ventana que recorre la imagen en cuestión.
- nn.BatchNorm2d: Normaliza un lote de imágenes o mapas de características, normaliza las activaciones de un lote de imágenes en cada canal. Estabiliza el entrenamiento y acelera la convergencia por medio de la reducción del cambio de covarianza interna.
- nn.Flatten: Convierte un tensor de múltiples dimensiones en un vector de una sola dimensión, por defecto mantiene la primera dimensión del lote intacta y aplanada desde la 1 en adelante.
- nn.CrossEntropyLoss: Se usa en problemas de clasificación con múltiples clases, recibe logits sin normalizar del modelo e índices de clase enteros como los objetivos.
- Tensor: estructura de datos parecida a un arreglo o una matriz multidimensional, con esta unidad se realizan cálculos, se almacenan entradas, salidas y parámetros de modelos.
- Receptive Field: Region de la imagen o datos de entrada que afectan los valores de neuronas específicas en una capa posterior. Es el área de la imagen que la neurona “ve”.
- Porque CNN necesitan menos parámetros que las MLP para procesar imágenes: La razón de esto se debe a que los CNN comparten pesos y conexiones locales, un filtro recorre la imagen y hace reuso de datos. En MLP esto no se da ya que conecta cada pixel de entrada con cada neurona independientemente y en CNN cada neurona ve una pequeña parte de la imagen.

Resultados

```
=== EXPLORACION Y PREPARACION DATASET ===
- Observaciones en entrenamiento: 60000
- Observaciones en prueba: 10000
  Total: 70000

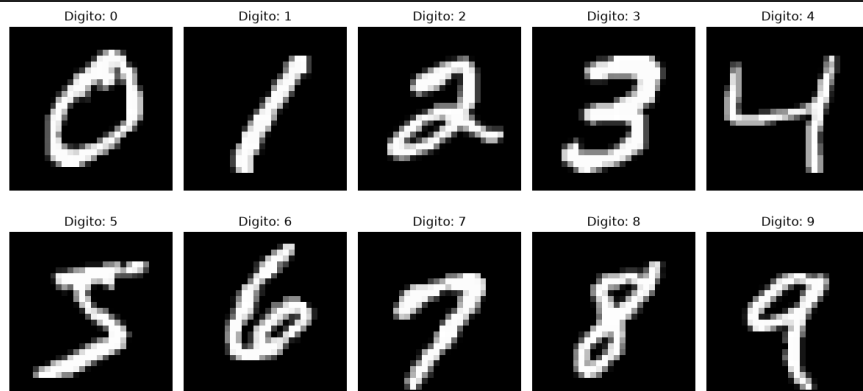
Total de clases: 10
['0 - zero', '1 - one', '2 - two', '3 - three', '4 - four', '5 - five', '6 - six', '7 - seven', '8 - eight', '9 - nine']

Distribucion clases:
Digito 0: 5923 imagenes (9.87%, diferencia del 1.28% al balance)
Digito 1: 6742 imagenes (11.24%, diferencia del 12.37% al balance)
Digito 2: 5958 imagenes (9.93%, diferencia del 0.70% al balance)
Digito 3: 6131 imagenes (10.22%, diferencia del 2.18% al balance)
Digito 4: 5842 imagenes (9.74%, diferencia del 2.63% al balance)
Digito 5: 5421 imagenes (9.04%, diferencia del 9.65% al balance)
Digito 6: 5918 imagenes (9.86%, diferencia del 1.37% al balance)
Digito 7: 6265 imagenes (10.44%, diferencia del 4.42% al balance)
Digito 8: 5851 imagenes (9.75%, diferencia del 2.48% al balance)
Digito 9: 5949 imagenes (9.92%, diferencia del 0.85% al balance)
```



```
--- Dimensiones y valores de pixeles ---
Dimensiones:
  torch.Size([1, 28, 28])
Alto: 28 píxeles
Ancho: 28 píxeles
Rango valores pixeles:
  - Max: 1.0
  - Min: 0.0
  - Media: 0.1376800686120987
  - Desv. Estandar: 0.3125477433204651
Total pixeles: 784

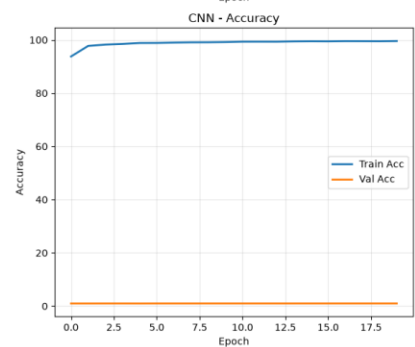
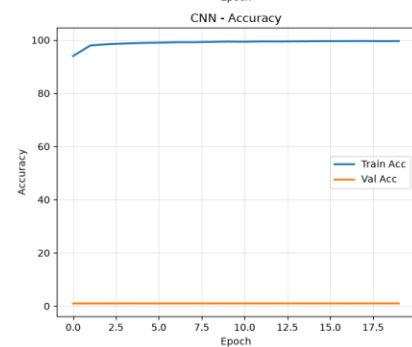
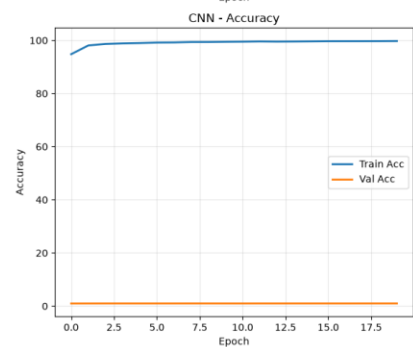
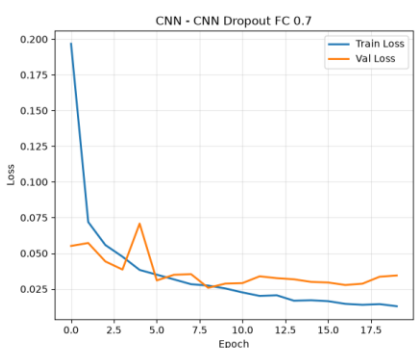
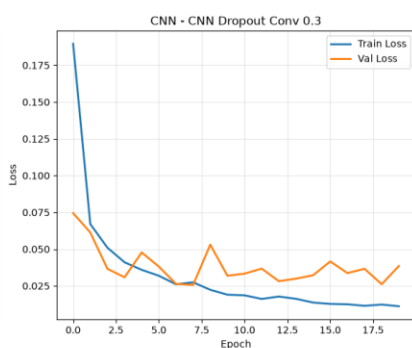
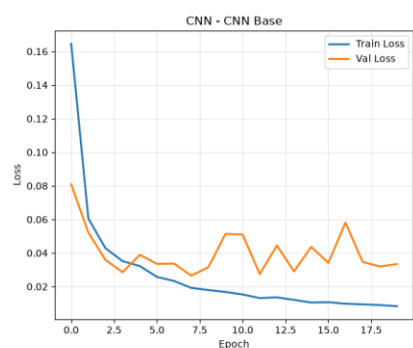
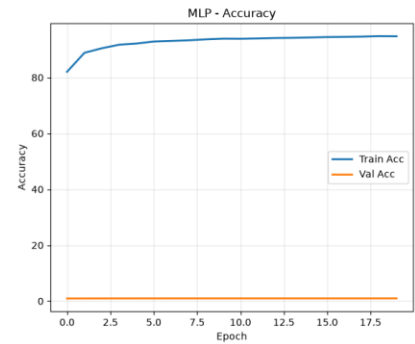
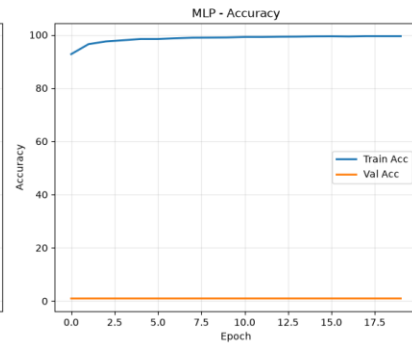
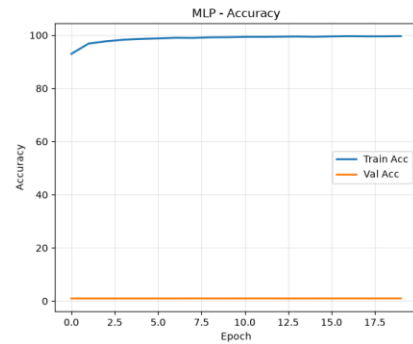
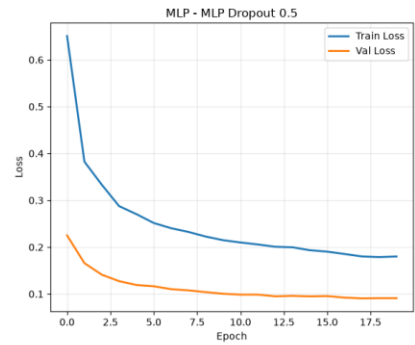
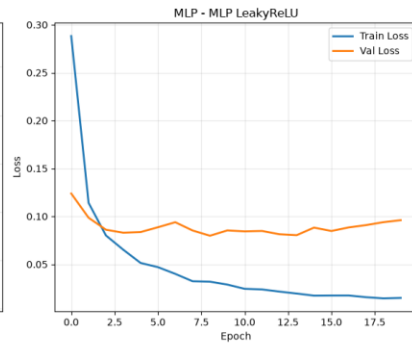
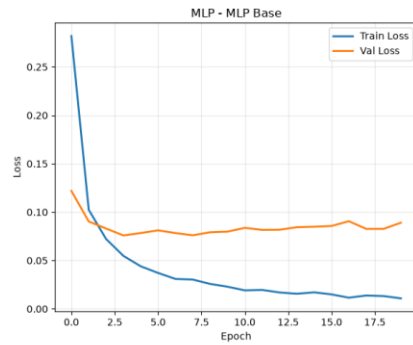
--- Ejemplos imagenes ---
Ej. digito 0: shape=torch.Size([1, 28, 28]), min=0.000, max=1.000
Ej. digito 1: shape=torch.Size([1, 28, 28]), min=0.000, max=1.000
Ej. digito 2: shape=torch.Size([1, 28, 28]), min=0.000, max=1.000
Ej. digito 3: shape=torch.Size([1, 28, 28]), min=0.000, max=1.000
Ej. digito 4: shape=torch.Size([1, 28, 28]), min=0.000, max=1.000
Ej. digito 5: shape=torch.Size([1, 28, 28]), min=0.000, max=1.000
Ej. digito 6: shape=torch.Size([1, 28, 28]), min=0.000, max=1.000
Ej. digito 7: shape=torch.Size([1, 28, 28]), min=0.000, max=1.000
Ej. digito 8: shape=torch.Size([1, 28, 28]), min=0.000, max=1.000
Ej. digito 9: shape=torch.Size([1, 28, 28]), min=0.000, max=1.000
```



```
--- Division Final ---  
Entrenamiento: 48000 imágenes  
Validación: 12000 imágenes  
Prueba: 10000 imágenes
```

```
--- Mejor MLP ---  
Config: {'name': 'Mejor MLP', 'hidden_layers': [256, 128, 64], 'activation': 'leaky_relu', 'dropout': 0.2, 'learning_rate': 0.001, 'batch_size': 64, 'epochs': 20}  
Parametros: 243,658  
Epoch 5/20:  
  -Perdida train: 0.10  
  -Accuracy train: 96.75%  
  -Perdida validacion: 0.08  
  -Accuracy validacion: 0.98%  
Epoch 10/20:  
  -Perdida train: 0.07  
  -Accuracy train: 97.78%  
  -Perdida validacion: 0.07  
  -Accuracy validacion: 0.98%  
Epoch 15/20:  
  -Perdida train: 0.05  
  -Accuracy train: 98.35%  
  -Perdida validacion: 0.07  
  -Accuracy validacion: 0.98%  
Epoch 20/20:  
  -Perdida train: 0.04  
  -Accuracy train: 98.62%  
  -Perdida validacion: 0.08  
  -Accuracy validacion: 0.98%  
Val Accuracy: 0.9795  
Val F1: 0.9795
```

```
--- CNN Mejor ---  
Config: {'name': 'CNN Mejor', 'dropout_conv': 0.2, 'dropout_fc': 0.5, 'learning_rate': 0.001, 'batch_size': 64, 'epochs': 20}  
Parametros: 620,234  
Epoch 5/20:  
  -Perdida train: 0.03  
  -Accuracy train: 98.97%  
  -Perdida validacion: 0.05  
  -Accuracy validacion: 0.99%  
Epoch 10/20:  
  -Perdida train: 0.02  
  -Accuracy train: 99.43%  
  -Perdida validacion: 0.04  
  -Accuracy validacion: 0.99%  
Epoch 15/20:  
  -Perdida train: 0.01  
  -Accuracy train: 99.58%  
  -Perdida validacion: 0.03  
  -Accuracy validacion: 0.99%  
Epoch 20/20:  
  -Perdida train: 0.01  
  -Accuracy train: 99.70%  
  -Perdida validacion: 0.04  
  -Accuracy validacion: 0.99%  
Val Accuracy: 0.9927  
Val F1: 0.9927
```



=== COMPARACION RESULTADOS ===

--- MLP ---

Configuracion	Params	Val Acc	Val F1	Precision	Recall
MLP Base	109,770	0.9785	0.9785	0.9785	0.9785
MLP Arquitectura 1	243,658	0.9812	0.9813	0.9813	0.9812
MLP Arquitectura 2	569,226	0.9818	0.9817	0.9818	0.9818
MLP LeakyReLU	109,770	0.9770	0.9770	0.9771	0.9770
MLP Dropout 0.2	109,770	0.9803	0.9803	0.9804	0.9803
MLP Dropout 0.5	109,770	0.9720	0.9720	0.9720	0.9720
MLP lr 0.0005	109,770	0.9778	0.9778	0.9779	0.9778
MLP lr 0.01	109,770	0.9811	0.9811	0.9811	0.9811
Mejor MLP	243,658	0.9795	0.9795	0.9796	0.9795

--- CNN ---

Configuracion	Params	Val Acc	Val F1	Precision	Recall
CNN Base	620,234	0.9924	0.9924	0.9924	0.9924
CNN Dropout Conv 0.1	620,234	0.9934	0.9934	0.9934	0.9934
CNN Dropout Conv 0.3	620,234	0.9928	0.9927	0.9928	0.9928
CNN Dropout FC 0.3	620,234	0.9899	0.9899	0.9900	0.9899
CNN Dropout FC 0.7	620,234	0.9910	0.9910	0.9911	0.9910
CNN LR 0.0005	620,234	0.9920	0.9920	0.9920	0.9920
CNN LR 0.01	620,234	0.9917	0.9917	0.9917	0.9917
CNN Mejor	620,234	0.9927	0.9927	0.9927	0.9927

=== EVALUACION MEJORES MODELOS (TEST) ===

Mejor MLP: MLP Arquitectura 2 (Val Acc: 0.98)

Mejor CNN: MLP Arquitectura 2 (Val Acc: 0.99)

--- MLP FINAL ---

Test Accuracy: 0.9830

Test Precision: 0.9830

Test Recall: 0.9830

Test F1: 0.9830

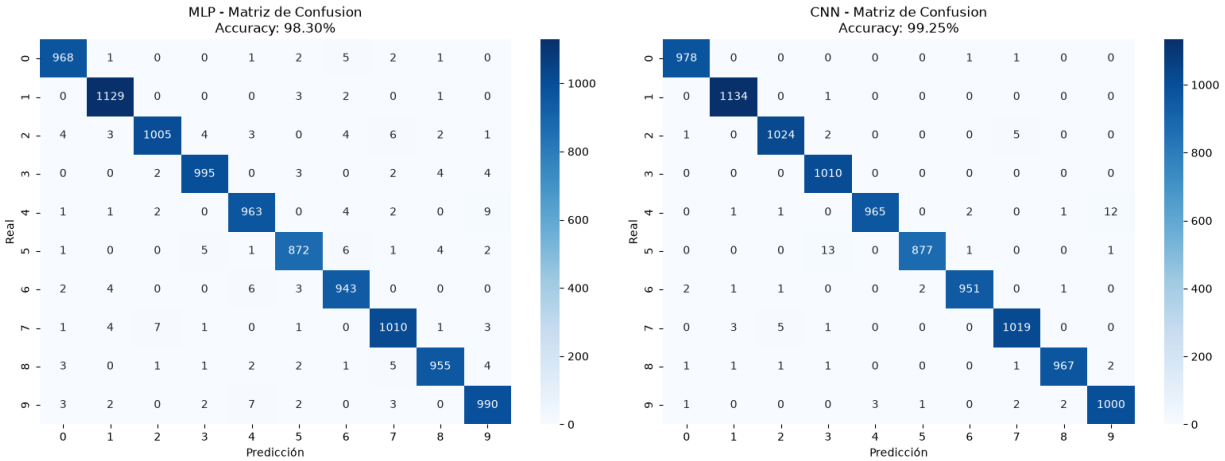
--- CNN FINAL ---

Test Accuracy: 0.9925

Test Precision: 0.9925

Test Recall: 0.9925

Test F1: 0.9925



```

=== UNDERFITTING Y OVERFITTING ===

--- MEJORES MODELOS ---

MLP - MLP Arquitectura 2:
  Train Loss final: 0.01
  Val Loss final: 0.08
  Loss Gap: 0.07
  Train Acc final: 99.67%
  Val Acc final: 0.98%
  Acc Gap: 98.69%
  Balance entre training y validacion

CNN - CNN Dropout Conv 0.1:
  Train Loss final: 0.01
  Val Loss final: 0.03
  Loss Gap: 0.02
  Train Acc final: 99.71%
  Val Acc final: 0.99%
  Acc Gap: 98.72%
  Balance entre training y validacion

=== RESUMEN ===

Métrica | MLP | CNN
-----|-----|-----
Test Accuracy | 0.9830 | 0.9925
Test Precision | 0.9830 | 0.9925
Test Recall | 0.9830 | 0.9925
Test F1-Score | 0.9830 | 0.9925
Parametros | 569,226 | 620,234
Mejor Config | MLP Arquitectura 2 | CNN Dropout Conv 0.1

```

Discusión y analisis

- ¿Qué cambio de hiperparámetro tuvo el mayor impacto positivo en las métricas de validación de cada arquitectura? ¿Y el mayor impacto negativo?

MLP:

Aumentar la arquitectura tuvo un mayor impacto positivo ya que se pudo observar un mayor cambio ya que al aumentar la cantidad de neuronas y capas se pueden aumentar la cantidad de variaciones. Creando una base de decisión más amplia.

Por otro lado el mayor impacto negativo fue de Dropout 0.5, esto debido a que el modelo solo tuvo una mitad de los parametros, llevando a que hubiera una sobre regularización. Esto provocó un underfitting, llevando a que el modelo no aprendiera lo suficiente con esta modificación.

CNN:

Dropout 0.1 fue la modificación con mejor impacto ya que regularizó los mapas y se mejoró la capacidad de generalización y manteniendo su balance con la retención de información. Por otro lado el Dropout de 0.3 lo impactó de forma negativa debido a que, en base a resultados, la regulación es baja llevando a que el modelo tiendiera a sobreajustarse.

- ¿Observaron overfitting o underfitting en alguna de sus iteraciones? ¿Cómo lo identificaron y qué hicieron para mitigarlo?

Por medio de las brechas entre los datos resultantes de entrenamiento y validación se logró identificar si había alguna de estas dos situaciones en los modelos, en los casos base de MLP se pudo ver que este era más sensible al overfitting debido a que tendía a

memorizar patrones más específicos. Un dropout moderado y ajuste de arquitectura pueden usarse para encontrar un balance entre su capacidad y su regularización.

CNN es más robusto ante el overfitting, cosa que se le puede atribuir a sus atributos de pooling y weight sharing, la mejora ligera en su dropout (0.1) demostró que este modelo ya estaba funcionando bastante bien y que este cambio simplemente lo refinó más.

- *¿La regularización mejoró el desempeño en validación? ¿Cuál método funcionó mejor para cada arquitectura y por qué creen que fue así?*

En el caso de MLP la regularización fue beneficiosa, si bien aun así se tiene que tener cuidado ya que también lo hace sensible. El rendimiento mejoró al modificar el dropout a 0.2, permitiendo una mejor generalización sin sacrificar su capacidad de representación, el dropout hace que no haya una dependencia específica entre ninguna neurona y así variando con que aprende.

En CNN aplicar un dropout también probó ser útil, específicamente con 0.1, siendo este la razón de que se obtuvo el mejor. Esta funcionó, aunque en CNN ya hay cierta regularización gracias a otros elementos en estas, pueden haber algunos beneficios de implementar un dropout, sin embargo se mantuvo una tasa baja debido a que las capas si son algo sensibles a este tipo de regularización.

- *Comparando el MLP y la CNN, ¿cuál obtuvo mejor desempeño en test? ¿Cómo se relaciona esa diferencia con la cantidad de parámetros de cada modelo y con la forma en que cada arquitectura procesa la información espacial de la imagen?*

En CNN, tomando en cuenta que tuvo mejores resultados en accuracy y una cantidad significativamente menor de errores. El rendimiento en CNN tiene que ver con que hace un mejor uso de sus parámetros, principalmente por su capacidad de compartir los pesos. Se mantiene la estructura de la imagen (2D) y se aplican filtros que revisan la imagen por cada capa, así detectando patrones en la imagen. Luego las capas de pooling dan invarianza a traslaciones y se mantienen las activaciones en la misma región, y así se establece una jerarquía de las características de las imágenes. Así se permite a la CNN reconocer los dígitos aunque la forma en la que están escritos varíe.

- *¿En qué tipo de errores se equivocan más el MLP y la CNN? Analicen las matrices de confusión de ambos modelos.*

El MLP se confunde más cuando se trata de números que tienen una forma parecida como el 9 y 4 o 7 y 1, al aplanar se pierde información que permite distinguir el trazo vertical más grande perteneciente al 4 o al 9. En el caso del CNN su error más común es la confusión entre el 7 y 9. Esto puede explicarse gracias a que el CNN suele ser más sensible ante la estructura espacial por lo que a veces se topaba con un 9 que tenía algunas partes que se trazaron de una forma similar a otras partes del 7, notoriamente cuando la escritura no era muy clara.

- *Si tuvieran que desplegar un modelo de producción para este problema, priorizando tanto exactitud como eficiencia (número de parámetros y tiempo de inferencia), ¿qué arquitectura elegirían y por qué?*

Desplegaría un modelo CNN por su robustez y que los errores son un poco más “humanos” por así decir, ya que eran más basados en escritura no tan buena y además de que hay un mejor equilibrio con la eficiencia y eficiencia. Haría uso de los parámetros que mejor se desempeñaron, o sea un dropout de 0.1, dropout fc 0.5 y por lo menos unas 3 capas.